

iOS登录页面开发实验报告

一、实验基本信息

- **实验编号:** d20301035102、d20301009202
- **实验名称:** 原生应用开发
- **学时分配:** 4学时
- **实验类型:** 验证型
- **负责平台:** iOS (SwiftUI)
- **姓名:** 丁陆
- **学号:** 2023210586
- **班级:** 软件工程23级1班
- **实验日期:** 2026年3月28日

实验目标

使用SwiftUI实现智慧校园登录模块，包括输入验证、页面跳转和数据传递功能，并添加个人信息管理功能。

项目结构

```
IosDemo/  
├── IosDemo/  
│   ├── IosDemoApp.swift    # 应用入口  
│   ├── ContentView.swift   # 登录页面  
│   ├── ProfileView.swift   # 个人信息页面  
│   └── Assets.xcassets/  
└── IosDemo.xcodeproj
```

核心代码实现

1. 应用入口 (IosDemoApp.swift)

```
@main  
struct IosDemoApp: App {  
    var body: some Scene {  
        WindowGroup {  
            LoginView()  
        }  
    }  
}
```

2. 登录页面 (ContentView.swift)

```
class LoginViewModel: ObservableObject {
    @Published var username = ""
    @Published var password = ""
    @Published var isLoading = false
    @Published var isLoggedIn = false

    func login() {
        guard !username.isEmpty else {
            alertMessage = "请输入用户名"
            showAlert = true
            return
        }

        guard !password.isEmpty else {
            alertMessage = "请输入密码"
            showAlert = true
            return
        }

        isLoading = true

        DispatchQueue.main.asyncAfter(deadline: .now() + 1.5) {
            if self.password == "123456" {
                self.isLoggedIn = true
            }
        }
    }
}
```

3. 个人信息页面 (ProfileView.swift)

```
struct UserProfile: Codable {
    var nickname = ""
    var studentId = ""
    var phone = ""
    var email = ""
    var gender = "保密"
    var bio = ""
}

class ProfileViewModel: ObservableObject {
    @Published var profile = UserProfile()
    @Published var isEditing = false

    func saveProfile() {
        if let encoded = try? JSONEncoder().encode(profile) {
            UserDefaults.standard.set(encoded, forKey: "userProfile")
        }
    }
}
```

PlantUML图表

1. 流程图（顺序图）

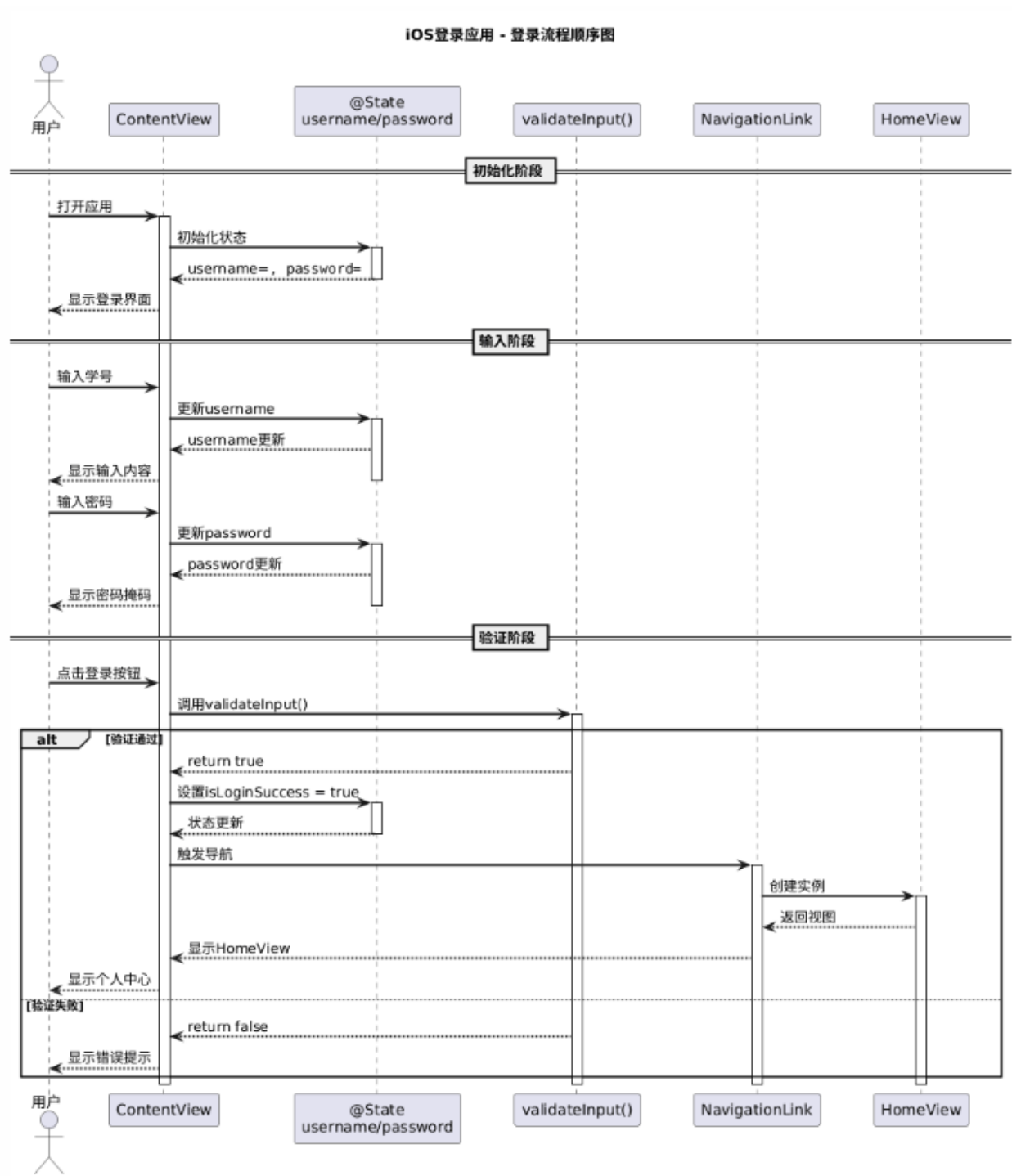


图1 iOS登录应用流程图（顺序图）

2. 类图

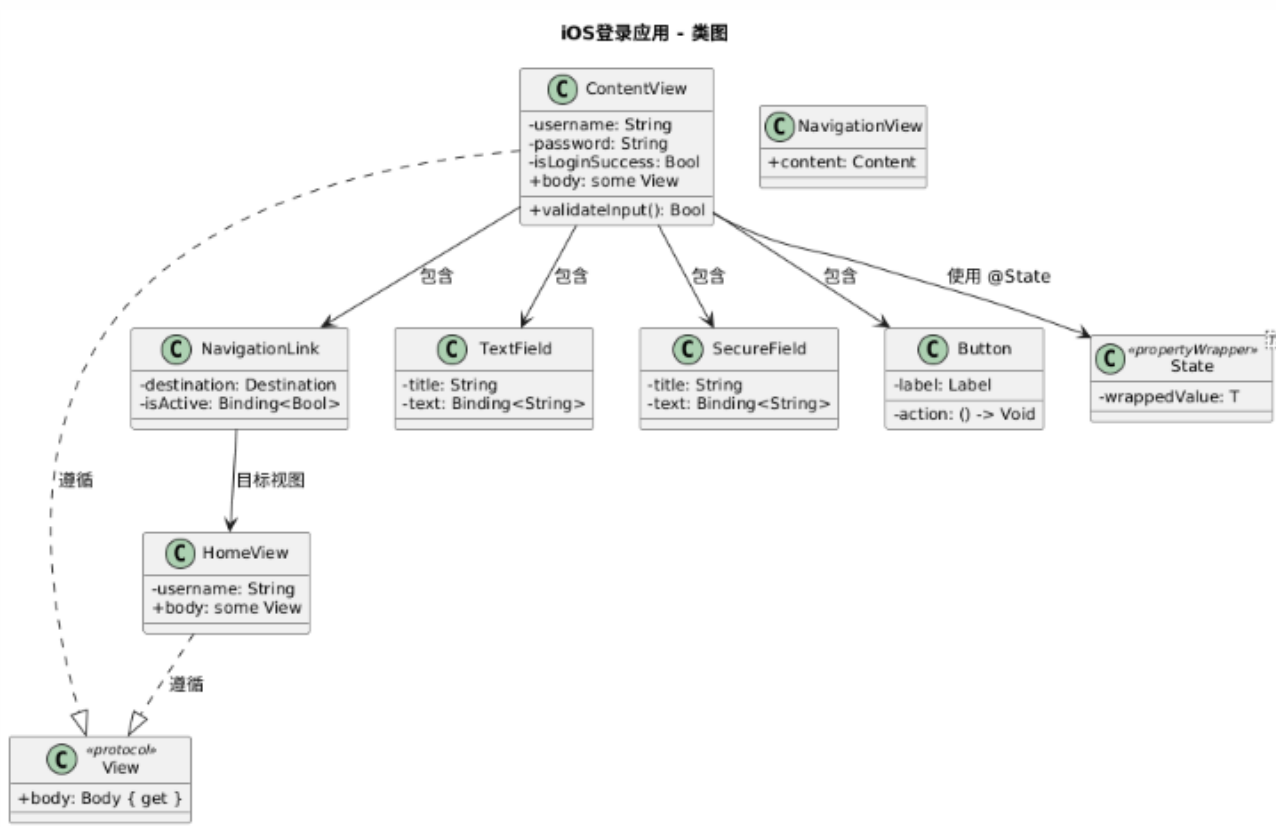


图2 iOS登录应用类图

3. 系统架构图

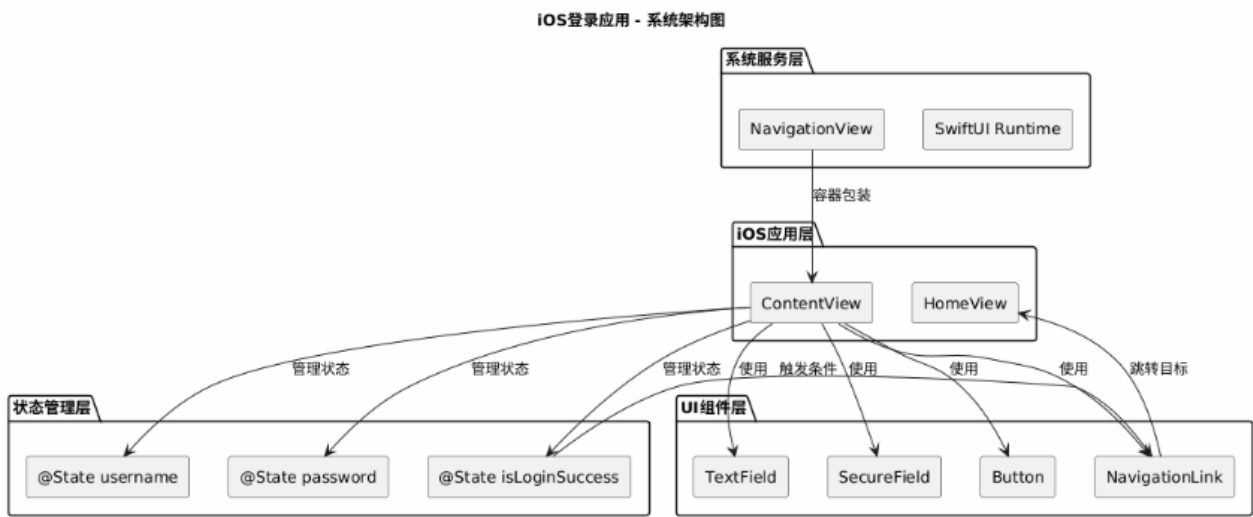


图3 iOS登录应用系统架构图

实验截图

截图1：登录页面

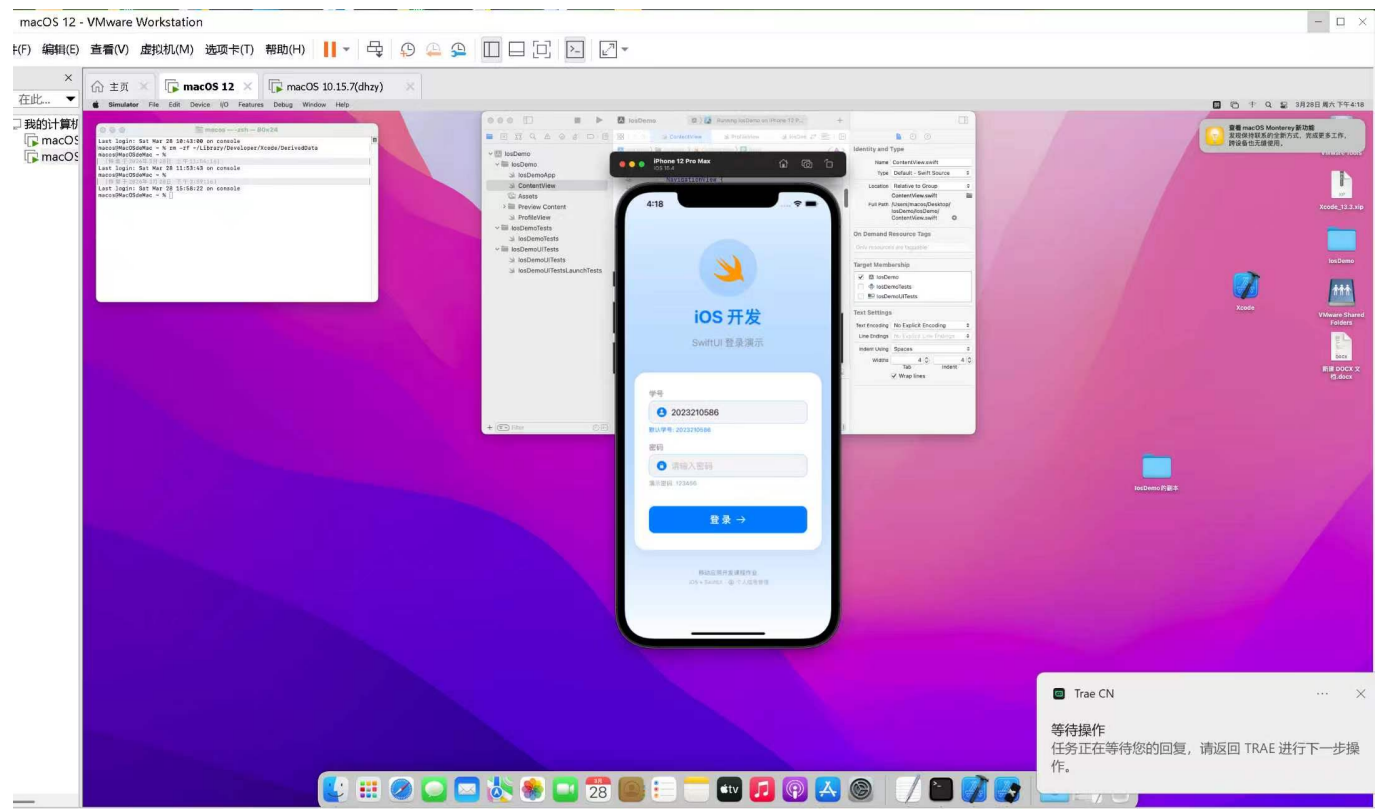


图4 iOS登录界面

截图2：输入验证

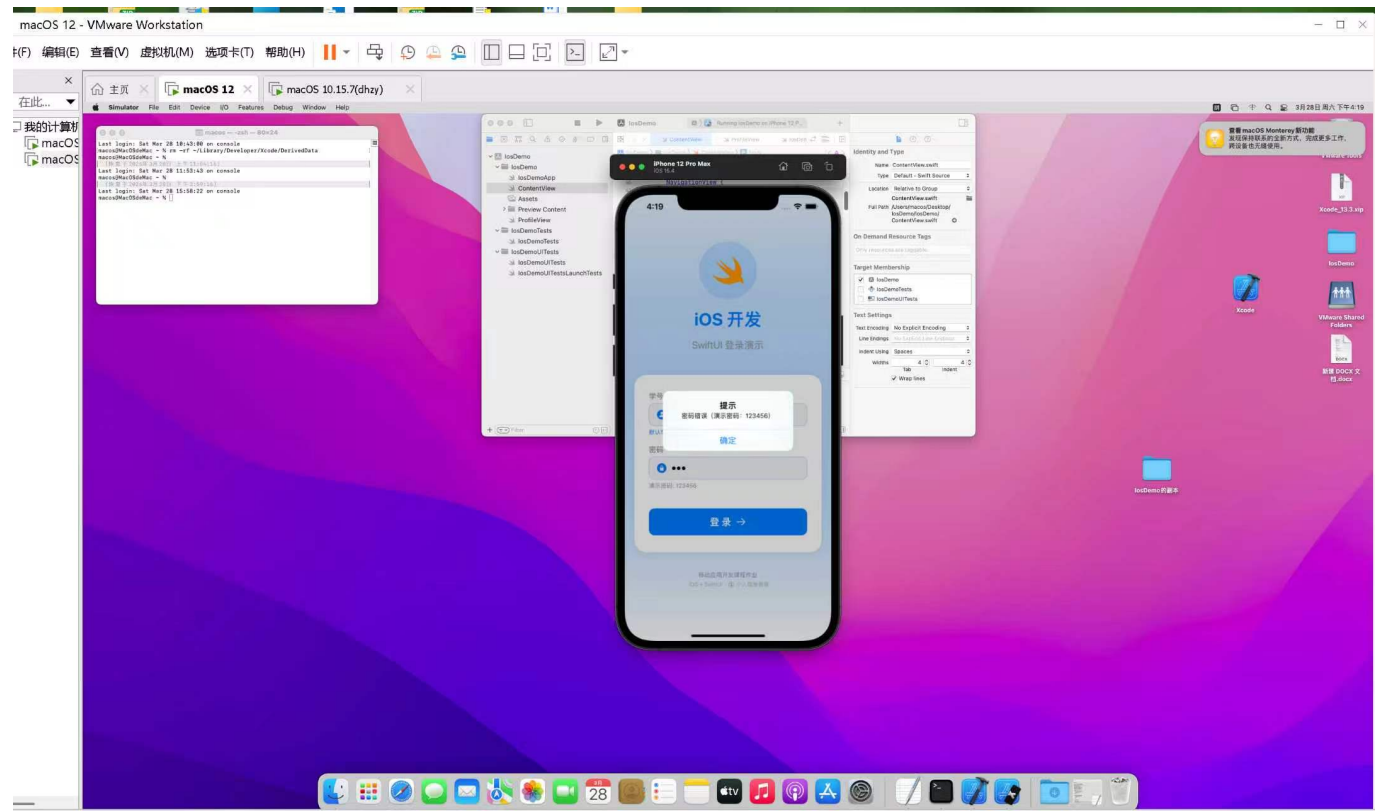


图5 iOS登录验证

截图3：登录成功

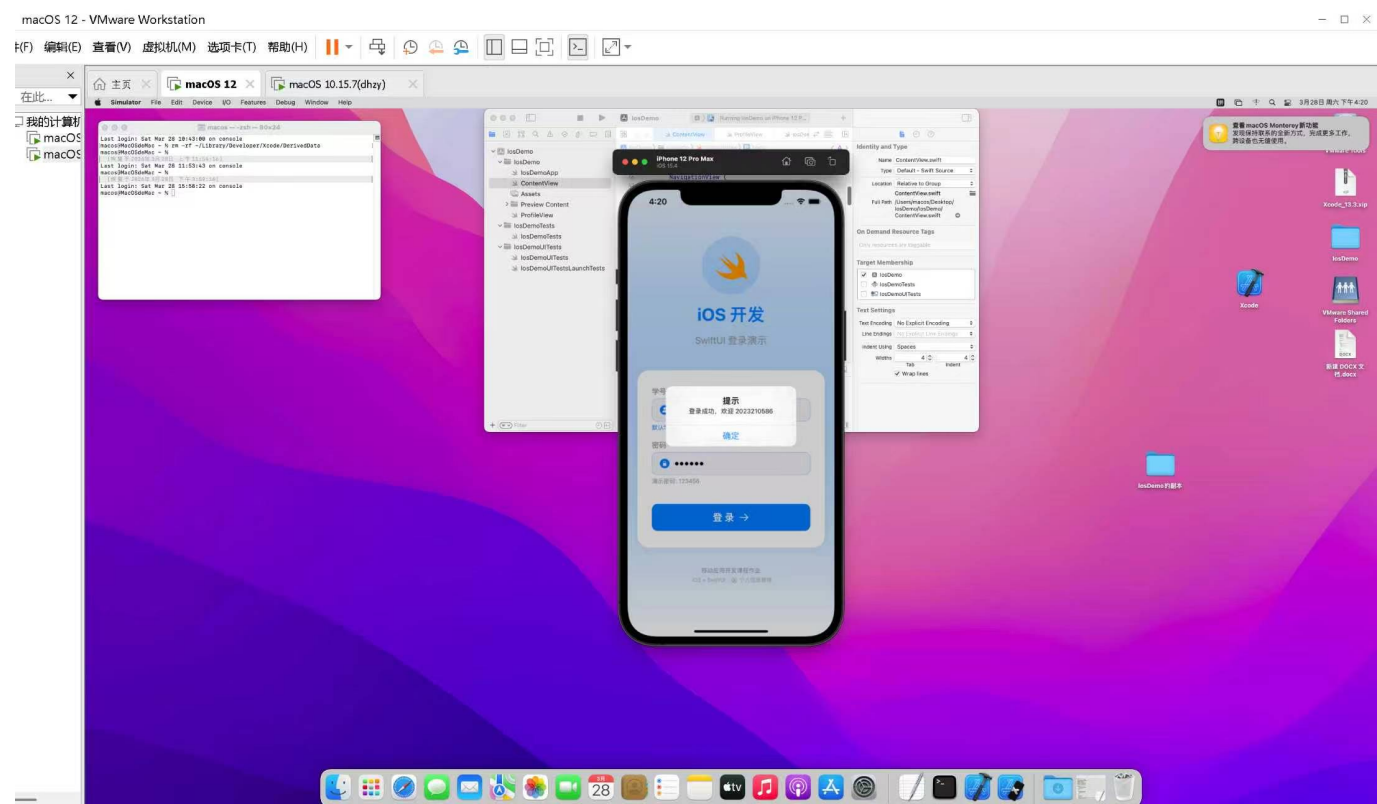


图6 iOS登录成功

测试用例

测试项	输入值	预期结果
空密码	用户名: "2023210586", 密码: ""	显示"请输入密码"错误
错误密码	用户名: "2023210586", 密码: "123"	显示"密码错误"提示
正确登录	用户名: "2023210586", 密码: "123456"	成功跳转到个人信息页面
编辑个人信息	修改昵称和学号	信息更新成功并保存
取消编辑	修改后点击取消	恢复原始信息

平台差异对比

对比项	Android	iOS
开发语言	Kotlin	Swift
UI框架	XML/Compose	SwiftUI
页面跳转	Intent	NavigationLink
数据传递	Intent.putExtra	参数传递
状态管理	成员变量	@Published/@State
布局方式	线性布局/约束布局	VStack/HStack

对比项	Android	iOS
表单编辑	EditText	TextField
数据持久化	SharedPreferences	UserDefaults
异步处理	Coroutines	DispatchQueue

实验总结

- 1. **功能实现**：完成了iOS登录页面的开发，包括输入验证、异步登录逻辑、页面跳转和个人信息管理功能
- 2. **技术要点**：MVVM架构模式、SwiftUI状态管理、Navigation导航、UserDefaults数据持久化
- 3. **学习收获**：理解了iOS原生开发的基本流程和SwiftUI框架的使用，掌握了MVVM架构在SwiftUI中的应用
- 4. **平台特色**：体验了iOS特有的界面设计风格和开发模式，学习了@Published和@ObservedObject的使用